

NBA Foul Predictor

Imports

```
import numpy as np
import polars as pl
from pyprojroot.here import here
from IPython.display import display ## just to remove an error

## To fix format of PDF output
pl.Config.set_tbl_cols(8);    ## show at most 8 columns
pl.Config.set_tbl_rows(10);   ## show at most 10 rows
```

Data Cleaning and File Merging

Combining the Referee Stats Files Into One Table

```
seasons = ['2017-2018', '2018-2019', '2019-2020', '2020-2021', '2021-2022', '2022-2023', '2023-2024', '2024-2025',
'2025-2026']

refList = []
for s in seasons:
    fileName = f'nba_referee_stats_{s}.csv'
    if s == '2025-2026':
        fileName = 'nba_referee_stats_2025_2026.csv' ## uses an underscore instead of dash between years

    refSeason = pl.read_csv(here(f'Data/{fileName}'))

    ## find foul differential column
    diffCol = [c for c in refSeason.columns if 'FOUL DIFFERENTIAL' in c][0]

    refSeason = (refSeason
        .rename({diffCol: 'foul_diff'})
        .rename({'TYPE': 'ROLE'}, strict = False) ## older files say TYPE
        .drop(['RANK', 'Unnamed: 13'], strict = False) ## empty columns
        .with_columns(Season = pl.lit(s)) ## adds Season if missing
    )
    refList.append(refSeason)

## Stack all seasons
refStats = pl.concat(refList, how = 'diagonal')

## Renaming columns
refStats = refStats.rename({
    'Season': 'season', 'REFEREE': 'referee', 'ROLE': 'role', 'GENDER': 'gender', 'EXPERIENCE (YEARS)': 'experience',
    'GAMES OFFICIATED': 'games', 'HOME TEAM WIN%': 'home_win_pct', 'HOME TEAM POINTS DIFFERENTIAL': 'home_pt_diff',
    'TOTAL POINTS PER GAME': 'points_per_game', 'CALLED FOULS PER GAME': 'fouls_per_game', 'FOUL% AGAINST ROAD TEAMS':
    'foul_pct_road', 'FOUL% AGAINST HOME TEAMS': 'foul_pct_home',
})

## Fixing percents
refStats = refStats.with_columns(
    pl.when(pl.col(c) > 1).then(pl.col(c) / 100).otherwise(pl.col(c)).alias(c)
    for c in ['foul_pct_road', 'foul_pct_home']
)
```

refStats

season	referee	role	experience	...	foul_pct_road	foul_pct_home	foul_diff	gender
str	str	str	i64	...	f64	f64	f64	str
"2017-2018"	"Ben Taylor"	"MAIN"	6	...	0.538	0.462	3.0	null
"2017-2018"	"Bennie Adams"	"MAIN"	22	...	0.498	0.502	-0.1	null
"2017-2018"	"Bill Kennedy"	"MAIN"	22	...	0.51	0.49	0.8	null
"2017-2018"	"Bill Spooner"	"MAIN"	28	...	0.513	0.488	1.0	null
"2017-2018"	"Brent Barnaky"	"MAIN"	8	...	0.493	0.507	-0.6	null
...	...	...	...	...	...	...	...	...
"2025-2026"	"Sean Corbin"	"CREW"	33	...	0.5	0.5	0.0	"MALE"
"2025-2026"	"Che Flores"	"CHIEF"	5	...	0.521	0.479	1.5	"FEMALE"
"2025-2026"	"Eric Lewis"	"CHIEF"	22	...	0.539	0.461	3.2	"MALE"
"2025-2026"	"Angel Kent"	"CREW"	1	...	0.447	0.553	-4.0	"FEMALE"
"2025-2026"	"Ben Taylor"	"CHIEF"	15	...	0.509	0.491	0.6	"MALE"

Combine CHIEF and CREW Rows

```
avgCols = ['home_win_pct', 'home_pt_diff', 'points_per_game', 'fouls_per_game', 'foul_pct_road', 'foul_pct_home',
'foul_diff']

refStats = (refStats
    .group_by(['season', 'referee']) ## referee rows for a season together
    .agg(
        experience = pl.col('experience').max(),
        gender = pl.col('gender').first(),
        games = pl.col('games').sum(), ## adding them
        *[(pl.col(c) * pl.col('games')).sum() / pl.col('games').sum()).alias(c) for c in avgCols] ## weighted average
    for every column in avgCols
    )
    .sort(['season', 'referee'])
)
refStats
```

season	referee	home_win_pct	home_pt_diff	...	foul_diff	experience	gender	games
str	str	f64	f64	...	f64	i64	str	i64
"2017-2018"	"Aaron Smith"	0.625	2.8	...	1.1	2	null	56
"2017-2018"	"Ben Taylor"	0.644823	5.606452	...	-0.73871	6	null	62
"2017-2018"	"Bennie Adams"	0.476615	0.5	...	0.176923	22	null	65
"2017-2018"	"Bill Kennedy"	0.588618	2.527941	...	1.289706	22	null	68
"2017-2018"	"Bill Spooner"	0.607588	2.690196	...	0.882353	28	null	51
...	...	...	...	...	...	...	...	...
"2025-2026"	"Tony Brothers"	0.621	3.9	...	-0.8	32	"MALE"	58
"2025-2026"	"Tre Maddox"	0.622222	5.044444	...	2.344444	16	"MALE"	45

season	referee	home_win_pct	home_pt_diff	...	foul_diff	experience	gender	games
str	str	f64	f64	...	f64	i64	str	i64
"2025-2026"	"Tyler Ford"	0.592797	4.215254	...	-0.452542	12	"MALE"	59
"2025-2026"	"Tyler Ricks"	0.55	-1.5	...	0.8	5	"MALE"	40
"2025-2026"	"Zach Zarba"	0.5667	1.526667	...	-0.826667	23	"MALE"	60

Cleaning Games File

```
games = pl.read_csv(here('Data/nba_games_ref_assignments.csv'))

games = (games
  ## Splitting season code (22025 --> game type 2, start year 2025)
  .with_columns(
    game_type = pl.col('season') // 10000,
    start_year = pl.col('season') % 10000,
  )
  ## Make it look like the referee table above ('2025-2026')
  .with_columns(
    season = pl.col('start_year').cast(pl.String) + '-' + (pl.col('start_year') + 1).cast(pl.String)
  )
  ## Keep only regular season games (2017-18)
  .filter(pl.col('game_type') == 2, pl.col('start_year') >= 2017)
  ## Fix team names
  .with_columns(
    pl.col(['home_team', 'away_team']).str.replace('L.A.', 'LA', literal=True)
  )
  ## Remove games listed twice
  .unique(subset = 'game_id', keep = 'first', maintain_order=True)
  ## Remove games missing referees
  .drop_nulls(subset = ['official1', 'official2', 'official3'])
  ## Drop 4th referee column (empty during reg season)
  .drop(['official4', 'official4_code', 'official4_JNum'])
)
games.shape
```

```
(10774, 21)
```

Matching Referee Stats to Games File - Fixing Names

```
## Referee Name fixed
nameFix = {
  'J.T. Orr': 'JT Orr',
  'Sharae Mitchell': 'Sha'Rae Mitchell', 'ShaRae Mitchell': 'Sha'Rae Mitchell',
  'Pat OConnell': 'Pat O'Connell', 'PAT OConnell': 'Pat O'Connell', 'PAT O'Connell': 'Pat O'Connell',
  'Cheryl Flores': 'Che Flores',
  'Jamahl Ralls': 'JD Ralls',
  'CP Painter': 'Chelisa Painter',
}

refStats = refStats.with_columns(pl.col('referee').replace(nameFix))

## One row per referee per game, attaching stats
gameRefs = (games
  .unpivot(index = ['game_id', 'season'], on = ['official1', 'official2', 'official3'],
    variable_name = 'slot', value_name = 'referee')
  .with_columns(pl.col('referee').replace(nameFix))
```

```

        .join(refStats, on = ['season', 'referee'], how = 'left')
    )
    ## Check - should be 0
    gameRefs['fouls_per_game'].null_count()

```

```
0
```

Average 3 Referees and Attach Games

```

crewStats = (gameRefs
    .group_by('game_id')
    .agg(
        crew_fouls_per_game = pl.col('fouls_per_game').mean(),
        crew_experience = pl.col('experience').mean(),
        crew_foul_diff = pl.col('foul_diff').mean(),
        crew_home_win_pct = pl.col('home_win_pct').mean(),
    )
)

## Attaching stats to games_TranslateTable
games = games.join(crewStats, on = 'game_id', how = 'left')
games.shape

```

```
(10774, 25)
```

For Code Below, We Don't Want Pull Data Every Time - Already Ran It Once

Pulling Each Season's Team Fouls

```

# import time
# from nba_api.stats.endpoints import leaguegamelog

# foulList = []
# for year in range(2017, 2026):
#     seasonCode = f'{year}-{str(year + 1)[-2:]}' ## nba_api wants '2017-18' not '2017-2018'

#     log = leaguegamelog.LeagueGameLog(
#         season = seasonCode,
#         season_type_all_star= 'Regular Season',
#         timeout = 60,
#     )

#     ## Turn result into polars table
#     result = log.get_dict()['resultSets'][0]
#     seasonLog = pl.DataFrame(result['rowSet'], schema = result['headers'], orient='row', infer_schema_length=None)
#     foulList.append(seasonLog.select('GAME_ID', 'TEAM_ABBREVIATION', 'PF', 'MIN'))
#     time.sleep(1) ## 1 second pause so we don't get blocked on NBA StopIteration

# ## Stack all seasons and save
# gameLogs = pl.concat(foulList)
# gameLogs.write_csv(here('Data/nba_team_game_logs.csv'))
# gameLogs

```

Reading Commented Out Above and Saving

```
gameLogs = pl.read_csv(here('Data/nba_team_game_logs.csv'))
gameLogs['MIN'].value_counts().sort('MIN')
```

MIN	count
i64	u32
240	20372
265	994
290	120
315	10
340	2

Total Fouls Per Game, Then Join

```
totalFouls = (gameLogs
    .with_columns(game_id = pl.col('GAME_ID').cast(pl.Int64)) ## 0021700001 --> 21700001
    .group_by('game_id')
    .agg(
        total_fouls = pl.col('PF').sum(),
        n_teams = pl.len(), ## about 2 every game
        game_minutes = pl.col('MIN').max(),
    )
)

## Attach to games
games = games.join(totalFouls, on = 'game_id', how = 'left')
games.shape
```

```
(10774, 28)
```

Checks

```
## Every game have exactly 2 teams
totalFouls['n_teams'].value_counts()
```

n_teams	count
u32	u32
2	10749

```
## How many games didn't get total_fouls
games['total_fouls'].null_count()
```

```
45
```

45 Missing Games (1/2 a Percent)

```
games.filter(pl.col('total_fouls').is_null()).group_by('season').len()
```

season	len
str	u32
"2019-2020"	45

Drop Games, and n_teams Column (Only Needed For One Check)

```
games = (games
        .drop_nulls(subset = 'total_fouls')
        .drop('n_teams')
)
games.shape
```

```
(10729, 27)
```

The 45 missing games are from 2019-20 (COVID Season) - league shut down

We first tried the team fouls file from NBAstuffer, but it stops at 2024-25, so all 2025-26 games were blank. Instead, we calculated each team's season foul average ourselves from the game logs, which covers every season.

Team Season Avg From gameLogs

```
teamSeasonFouls = (gameLogs
    ## Get season from game ID
    .with_columns(start_year = 2000 + (pl.col('GAME_ID') // 100000) % 100)
    .with_columns(
        season = pl.col('start_year').cast(pl.String) + '-' + (pl.col('start_year') + 1).cast(pl.String)
    )
    ## Average fouls per game for each team in season
    .group_by(['season', 'TEAM_ABBREVIATION'])
    .agg(team_fouls = pl.col('PF').mean())
)
teamSeasonFouls
```

season	TEAM_ABBREVIATION	team_fouls
str	str	f64
"2017-2018"	"HOU"	19.47561
"2017-2018"	"IND"	18.829268
"2017-2018"	"NOP"	19.146341
"2018-2019"	"LAC"	23.329268
"2018-2019"	"DAL"	20.121951
...	...	...
"2024-2025"	"HOU"	18.987805
"2025-2026"	"GSW"	19.609756
"2025-2026"	"BKN"	21.073171
"2025-2026"	"MEM"	20.487805
"2025-2026"	"CHA"	19.073171

Attach Team Foul Averages (Home and Away)

```
games = (games
    .join(teamSeasonFouls.rename({'TEAM_ABBREVIATION': 'home_team_abbr', 'team_fouls': 'home_team_fouls'}), on =
    ['season', 'home_team_abbr'], how = 'left')
    .join(teamSeasonFouls.rename({'TEAM_ABBREVIATION': 'away_team_abbr', 'team_fouls': 'away_team_fouls'}), on =
    ['season', 'away_team_abbr'], how = 'left')
)
games.select(pl.col('home_team_fouls', 'away_team_fouls').null_count())
```

home_team_fouls	away_team_fouls
u32	u32
0	0

All files combined into one table (games). There's 10,729 games, each with crew stats, team foul avgs, and total_fouls (target number).

Removing Unuseful Columns

```
gamesClean = (games
    .with_columns(game_date = pl.col('game_date').str.to_date('%m/%d/%Y')) ## Changing date format
    ## Overtime periods and late season
    .with_columns(
        overtime = ((pl.col('game_minutes') - 240) / 25).cast(pl.Int64),
        days_left = (pl.col('game_date').max().over('season') - pl.col('game_date')).dt.total_days(),
    )
    .with_columns(late_season = pl.col('days_left') <= 14)
    .select(
        'game_id', 'game_date', 'season', 'home_team_abbr', 'away_team_abbr', 'official1', 'official2', 'official3',
        'crew_fouls_per_game', 'crew_experience', 'crew_foul_diff', 'crew_home_win_pct', 'home_team_fouls', 'away_team_fouls',
        'overtime', 'late_season', 'total_fouls' ## Only keeping useful columns
    )
    .sort('game_date')
)
gamesClean.write_parquet(here('Data/nba_games_clean.parquet'))
gamesClean
```

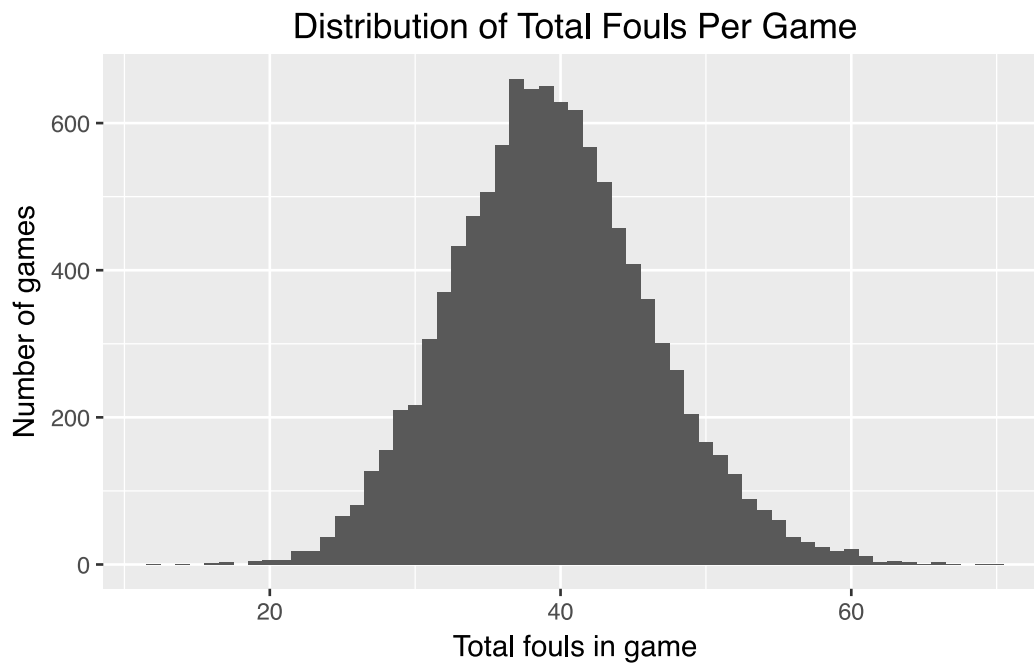
game_id	game_date	season	home_team_abbr	...	away_team_fouls	overtime	late_season	total_fouls
i64	date	str	str	...	f64	i64	bool	i64
21700001	2017-10-17	"2017-2018"	"CLE"	...	19.731707	0	false	49
21700002	2017-10-17	"2017-2018"	"GSW"	...	19.47561	0	false	41
21700003	2017-10-18	"2017-2018"	"DET"	...	17.182927	0	false	36
21700004	2017-10-18	"2017-2018"	"IND"	...	20.585366	0	false	50
21700005	2017-10-18	"2017-2018"	"ORL"	...	20.097561	0	false	38
...	...	...	...	...	...	...	...	...
22501196	2026-04-12	"2025-2026"	"OKC"	...	20.621951	0	true	17
22501197	2026-04-12	"2025-2026"	"SAS"	...	19.134146	0	true	34
22501198	2026-04-12	"2025-2026"	"LAL"	...	21.134146	0	true	34
22501199	2026-04-12	"2025-2026"	"LAC"	...	19.609756	0	true	45
22501200	2026-04-12	"2025-2026"	"POR"	...	19.682927	0	true	39

Exploratory Data Analysis (EDA)

Histogram of Total Fouls

```
from plotnine import *
import pyarrow

(ggplot(gamesClean, aes(x = 'total_fouls'))
 + geom_histogram(binwidth=1)
 + labs(x = 'Total fouls in game', y = 'Number of games', title = 'Distribution of Total Fouls Per Game')
 )
```



Look at Unusual Games

```
gamesClean['total_fouls'].describe()
```

statistic	value
str	f64
"count"	10729.0
"null_count"	0.0
"mean"	39.452698
"std"	6.885991
"min"	12.0
"25%"	35.0
"50%"	39.0
"75%"	44.0
"max"	70.0

Games With Fewest and Most Fouls

```
## Lowest
gamesClean.sort('total_fouls').select('game_date', 'home_team_abbr', 'away_team_abbr', 'total_fouls').head(10)
```

game_date	home_team_abbr	away_team_abbr	total_fouls
date	str	str	i64
2024-04-09	"MIL"	"BOS"	12
2026-01-12	"IND"	"BOS"	14
2025-04-11	"PHX"	"SAS"	16
2025-04-13	"SAC"	"PHX"	16
2024-03-02	"LAL"	"DEN"	17
2026-02-19	"GSW"	"BOS"	17
2026-04-12	"OKC"	"PHX"	17
2024-04-09	"UTA"	"DEN"	19
2024-04-12	"DAL"	"DET"	19
2024-04-14	"CLE"	"CHA"	19

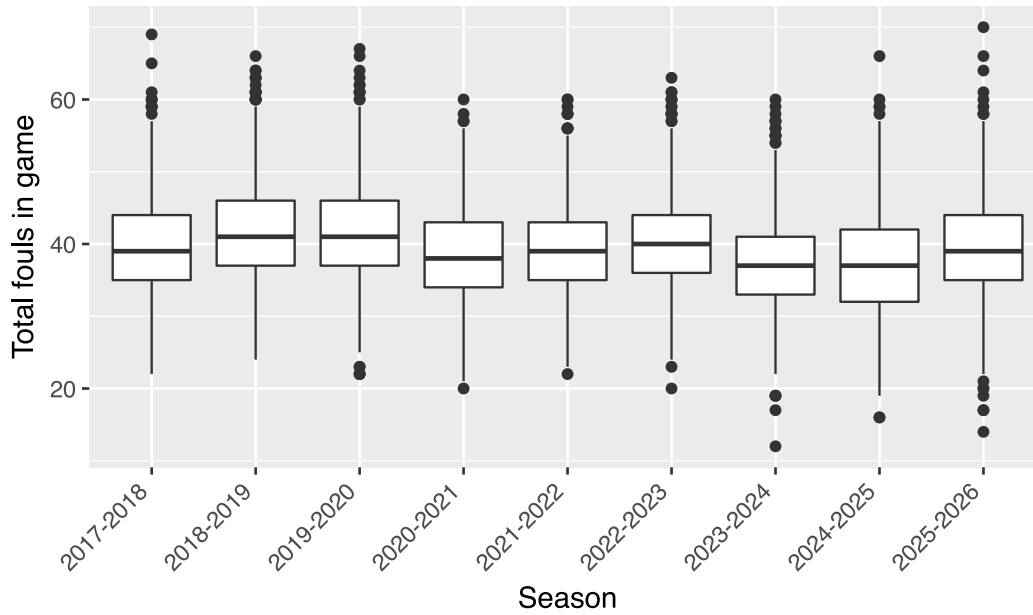
```
## Highest
gamesClean.sort('total_fouls', descending=True).select('game_date', 'home_team_abbr', 'away_team_abbr', 'total_fouls').head(10)
```

game_date	home_team_abbr	away_team_abbr	total_fouls
date	str	str	i64
2025-10-23	"IND"	"OKC"	70
2017-11-29	"PHI"	"WAS"	69
2020-07-31	"DAL"	"HOU"	67
2018-11-09	"PHI"	"CHA"	66
2019-10-26	"PHX"	"LAC"	66
2024-10-25	"TOR"	"PHI"	66
2025-11-05	"LAL"	"SAS"	66
2017-12-31	"HOU"	"LAL"	65
2019-01-02	"PHX"	"PHI"	64
2019-03-01	"ATL"	"CHI"	64

Fouls Over Time

```
(ggplot(gamesClean, aes(x = 'season', y = 'total_fouls'))
+ geom_boxplot()
+ labs(x = 'Season', y = 'Total fouls in game', title = 'Total Fouls Per Game by Season')
+ theme(axis_text_x = element_text(rotation = 45, ha = 'right'))
)
```

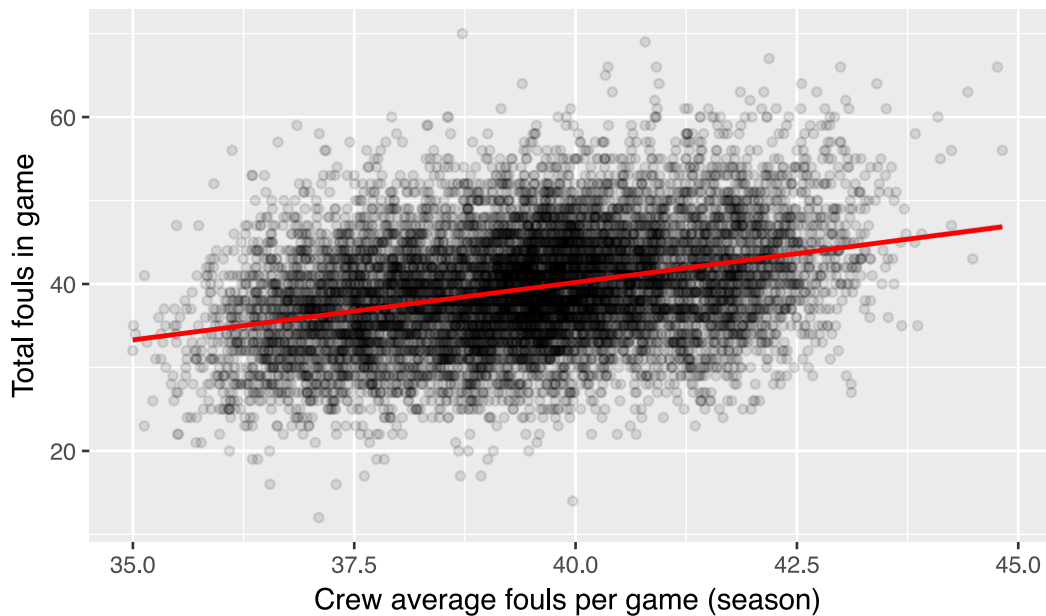
Total Fouls Per Game by Season



Does The Crew Actually Matter

```
(ggplot(gamesClean, aes(x = 'crew_fouls_per_game', y = 'total_fouls'))
+ geom_point(alpha = 0.1)
+ geom_smooth(method = 'lm', color = 'red')
+ labs(x = 'Crew average fouls per game (season)', y = 'Total fouls in game', title = 'Crew Foul Tendency vs.
Total Fouls')
)
```

Crew Foul Tendency vs. Total Fouls



Correlations

```
corrTable = (gamesClean
    .select('total_fouls', 'crew_fouls_per_game', 'crew_experience', 'crew_foul_diff',
        'crew_home_win_pct', 'home_team_fouls', 'away_team_fouls', 'overtime')
    .corr()
    .select(pl.all().round(2))
    ## Short names so the table fits on the page
    .rename({
        'total_fouls': 'total', 'crew_fouls_per_game': 'crew_fouls', 'crew_experience': 'crew_exp',
        'crew_foul_diff': 'crew_diff', 'crew_home_win_pct': 'crew_hwp', 'home_team_fouls': 'home_fouls',
        'away_team_fouls': 'away_fouls', 'overtime': 'OT',
    })
)

## Add a label column so each row has a name too
## Show all columns for just this table
with pl.Config(tbl_cols = 10):
    display(corrTable.insert_column(0, pl.Series('variable', corrTable.columns)))
```

variable	total	crew_fouls	crew_exp	crew_diff	crew_hwp	home_fouls	away_fouls	OT
str	f64	f64	f64	f64	f64	f64	f64	f64
"total"	1.0	0.34	-0.04	0.05	0.05	0.3	0.29	0.21
"crew_fouls"	0.34	1.0	-0.12	0.19	0.24	0.48	0.48	0.04
"crew_exp"	-0.04	-0.12	1.0	0.04	0.01	-0.06	-0.07	-0.02
"crew_diff"	0.05	0.19	0.04	1.0	0.24	0.07	0.15	-0.01
"crew_hwp"	0.05	0.24	0.01	0.24	1.0	0.12	0.15	0.0
"home_fouls"	0.3	0.48	-0.06	0.07	0.12	1.0	0.25	0.02
"away_fouls"	0.29	0.48	-0.07	0.15	0.15	0.25	1.0	0.0
"OT"	0.21	0.04	-0.02	-0.01	0.0	0.02	0.0	1.0

```
from sklearn.linear_model import LinearRegression
from scipy import stats
import matplotlib.pyplot as plt

predictors = [
    "crew_fouls_per_game", "crew_experience", "crew_foul_diff",
    "crew_home_win_pct", "home_team_fouls", "away_team_fouls",
    "overtime", "late_season"
]

model_data = gamesClean.select(predictors + ["total_fouls"]).drop_nulls()
X = model_data.select(predictors).to_numpy().astype(float)
y = model_data["total_fouls"].to_numpy().astype(float)

model = LinearRegression().fit(X, y)
predicted = model.predict(X)
residuals = y - predicted

fig, axes = plt.subplots(1, 3, figsize=(14, 4))

# Constant variance: look for a similar vertical spread across predictions.
```

```

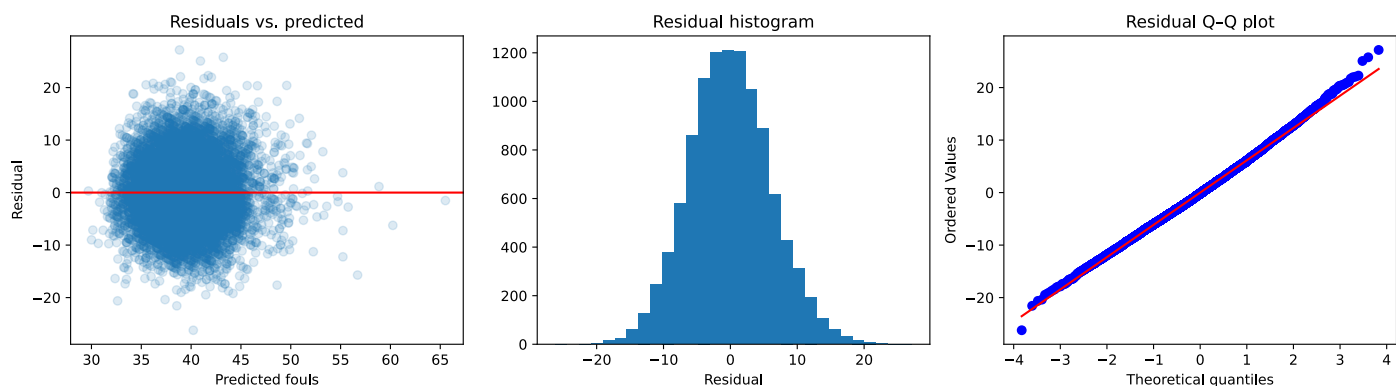
axes[0].scatter(predicted, residuals, alpha=0.15)
axes[0].axhline(0, color="red")
axes[0].set(xlabel="Predicted fouls", ylabel="Residual",
            title="Residuals vs. predicted")

# Normality: inspect the residual distribution and Q-Q plot.
axes[1].hist(residuals, bins=30)
axes[1].set(xlabel="Residual", title="Residual histogram")
stats.probplot(residuals, dist="norm", plot=axes[2])
axes[2].set_title("Residual Q-Q plot")

plt.tight_layout()
plt.show()

# Identifiability: rank should equal the number of model columns.
design_matrix = np.column_stack([np.ones(len(X)), X])
print("Matrix rank:", np.linalg.matrix_rank(design_matrix))
print("Number of model columns:", design_matrix.shape[1])
print("Condition number:", np.linalg.cond(design_matrix))

```



```

Matrix rank: 9
Number of model columns: 9
Condition number: 1419.2908968406418

```

Constant Variance

The residuals stay centered around zero, with no clear funnel pattern across most predicted foul totals. The spread looks reasonably similar in the range where most games fall, though there are a few games at the highest predictions, so constant variance appears approximately reasonable.

Normality

The residual histogram is roughly bell shaped, and most Q-Q points follow the reference line. The ends of the Q-Q plot bend away from the line, so the normality is approximate, with some departures in the tails.

Identifiability

The model matrix has full rank (9 out of 9 columns), so the coefficients can be estimated separately. The condition number is about 1,419, which suggests some numerical sensitivity, although differences in predictor scales can also raise this number. Identifiability is satisfied in the strict full rank sense, but we should be cautious when interpreting individual coefficients.

Linearity (The mapping function from the predictors to the outcome is an additive, linear function of the parameters)

lets run the code to show a scatterplot of residuals vs. fitted values

```
import statsmodels.formula.api as smf
```

```

# Fit another model (for teaching purposes)
fit_02 = smf.ols("total_fouls ~ crew_fouls_per_game + crew_experience + crew_foul_diff + crew_home_win_pct +
home_team_fouls + away_team_fouls + overtime + late_season",
data = gamesClean
).fit()

# Save the residuals and fitted values
gamesClean = (gamesClean
.with_columns(residuals_02 = pl.Series(fit_02.resid))
.with_columns(fitted_values_02 = pl.Series(fit_02.fittedvalues))
)

(ggplot(
  gamesClean,
  aes(
    x = 'residuals_02',
    y = 'fitted_values_02'
  )
)
+ geom_point(alpha = 0.25)
+ geom_smooth(
  method = 'lowess',
  color = 'red'
)
+ labs(
  title =
    'Linearity: Residuals vs. Fitted Values',
  subtitle =
    'Look for a roughly horizontal line'
)
)

```

```

/Users/whitneyholt/Desktop/NBAFoulPredictor_ML/.venv/lib/python3.14/site-packages/plotnine/stats/smoothers.py:342:
PlotnineWarning: Confidence intervals are not yet implemented for lowess smoothings.

```

Looking at the scatterplot of residuals vs. fitted values, we can see the line of best fit is almost completely horizontal. This tells us the mapping function from the predictors to the outcome is an additive, linear function of the parameters and satisfies the linearity assumption.

Independence (Observations are independent of each other)

In order for us to determine if our observations are independent of each other, we will create a sequential residual graph

```

# Add sequential observation numbers
gamesClean = gamesClean.with_row_index("observation")

#sequential residual graph
(ggplot(
  gamesClean,
  aes(x = 'observation', y = 'residuals_02')
)
+ geom_line(alpha = 0.5)
+ geom_hline(yintercept = 0, color = 'red')
+ labs(
  title =
    'Independence: Sequential Residuals',
  subtitle =
    'Look for a trend in mean or variance'
)
)

```

```
)  
)
```

As we can see from the Sequential Residuals plot, there is no clear order or trends we can see. It seems to be completely random, which tells me that our observations are independent of each other and the independence assumption is validated.

Milestone 08

Imports

```
import numpy as np  
import polars as pl  
import statsmodels.api as sm  
from pyprojroot.here import here  
from sklearn.model_selection import train_test_split  
  
gamesClean = pl.read_parquet(here('Data/nba_games_clean.parquet'))  
  
X = gamesClean.select(  
    'crew_fouls_per_game', 'crew_experience', 'crew_foul_diff', 'crew_home_win_pct', 'home_team_fouls',  
    'away_team_fouls', 'overtime', 'late_season', 'season'  
)  
y = gamesClean.select('total_fouls')  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Log Transformation

Our team decided that based on our data being relatively bell-shaped, we will not be needing to do a log transformation

Dummy Coding

```
X_train = (X_train  
    .with_columns(pl.col('late_season').cast(pl.Int64))  
    .to_dummies(columns = ['season'], drop_first = False)  
    .select(pl.exclude('season_2017-2018'))  
    .rename(lambda c: c.replace('-', '_'))  
)  
X_train.shape
```

Ordinary Least Squares (OLS) Model

```
# Add constant for intercept  
X_train = sm.add_constant(X_train.to_pandas())  
  
# Fit a model  
fit_05 = sm.OLS(y_train.to_pandas(), X_train).fit()  
fit_05.summary()
```

Milestone 09

Running Bayesian model

```
import pytensor  
pytensor.config.cxx = ""
```

```

import bambi as bmb
import arviz as az
import pandas as pd

# Specify predictors
predictors = [
    'crew_fouls_per_game', 'crew_experience', 'crew_foul_diff', 'crew_home_win_pct', 'home_team_fouls',
    'away_team_fouls',
    'overtime', 'late_season', 'season_2018_2019', 'season_2019_2020', 'season_2020_2021', 'season_2021_2022',
    'season_2022_2023', 'season_2023_2024', 'season_2024_2025', 'season_2025_2026'
]

# combine 2 training sets
nba_train = pd.concat(
    [
        y_train.to_pandas() if isinstance(y_train, pl.DataFrame) else y_train,
        X_train.drop(columns='const', errors='ignore')
        if isinstance(X_train, pd.DataFrame)
        else X_train.to_pandas()
    ],
    axis=1
)

# Bayesian linear regression model
ba_model = bmb.Model(
    'total_fouls ~ ' + ' + '.join(predictors),
    data = nba_train
)

# Specify priors
priors_dict = {
    'Intercept': bmb.Prior('Uniform', lower = -100, upper = 100),
    **{term: bmb.Prior('Uniform', lower = -100, upper = 100) for term in
predictors},
    'sigma': bmb.Prior('Exponential', lam = 1)
}

# setting the priors
ba_model.set_priors(priors = priors_dict)

# OLS confidence intervals and p-values
confidence_intervals = fit_05.conf_int()
confidence_intervals.columns = ['CI Lower', 'CI Upper']

ols_results = pd.concat(
    [
        fit_05.params.rename('Estimate'),
        confidence_intervals,
        fit_05.pvalues.rename('p-value')
    ],
    axis=1
)
print(ols_results.round(3))

# Fit a Bayesian linear regression
ba_fit = ba_model.fit(
    chains=4,
    cores=1,
    random_seed=123
)

```

```
# fit summary and intervals
fit_05.summary()
az.summary(ba_fit, ci_kind = 'hdi', ci_prob = 0.95)
```

Param Estimates and Interpretations Confidence Intervals

First, it's important to note a couple of things. We are using `total_fouls` as our dependent variable in this model. Another important number is the R^2 value, which is 0.265. This tells us that the predictors in the model explain about 26.5% of the variation in `total_fouls` in the training data.

Intercept (-111.558):

- This is the predicted total fouls when every predictor is 0. However, this is very unlikely, so we won't interpret it.
 - The 95% confidence interval is -118.94 to -104.18, and the 95% credible interval is -120 to -100. Both exclude zero, but because the intercept has no real-world meaning, its significance doesn't tell us anything useful.

Crew Variables:

- `crew_fouls_per_game` (2.592), for every 1 extra foul the crew usually calls per game (their season average), there are about 2.6 more total fouls in the game, holding all other variables fixed.
 - *Confidence Interval*: we are 95% confident that each extra usual crew foul increases total fouls by between 2.43 and 2.76, holding all other variables fixed.
 - *Credible Interval*: with probability 0.95, each extra usual crew foul increases total fouls by between 2.4 and 2.8, holding all other variables fixed
 - Both intervals exclude zero ($p < 0.001$), so this predictor is statistically significant. It is also practically significant: its effect is more than twice as large as either team variable (about 1.2), which supports our idea that referees matter more than teams.
- `crew_experience` (0.004): One more year of average crew experience changes total fouls by almost nothing, holding all other variables fixed.
 - The 95% confidence interval is -0.025 to 0.034 ($p = 0.770$), and the 95% credible interval is -0.024 to 0.034.
 - Both include zero, so this predictor is not statistically significant.
- `crew_foul_diff` (0.091): Crews that call more fouls on road teams than home teams don't clearly change total fouls, holding all other variables fixed.
 - The 95% confidence interval is -0.204 to 0.387 ($p = 0.546$), and the 95% credible interval is -0.22 to 0.38.
 - Both include zero, so this predictor is not statistically significant.
- `crew_home_win_pct` (0.224): This is a proportion (0 to 1), so a one unit increase means going from 0% to 100% home team wins. Even that huge change has no clear effect on total fouls, holding all other variables fixed.
 - The 95% confidence interval is -2.826 to 3.275 ($p = 0.885$), and the 95% credible interval is -2.8 to 3.3. The interval is wide because of the 0-to-1 scale.
 - Both include zero, so this predictor is not statistically significant.
- Not being statistically significant doesn't mean these three variables should be removed from the model. We keep them as controls, and knowing that experience and home bias don't affect total fouls is a useful finding.

Team Variables:

- `home_team_fouls` (1.211), this means that if a home team fouls 1 more time per game, that the game has about 1.2 more total fouls. The Bayesian 95% credible interval is approximately [1.1, 1.3], meaning that, given the model and prior assumptions, there is a 95% posterior probability that the `home_team_fouls` coefficient falls between 1.1 and 1.3. The frequentist 95% confidence interval for the coefficient is [1.107, 1.314], meaning that the estimated increase in total fouls associated with `home_team_fouls` is between 1.107 and 1.314.
- `away_team_fouls` (1.185), has the same idea, that if they foul 1 more time per game, that the game will have 1.2 more total fouls. The Bayesian 95% credible interval is approximately [1.082, 1.288], meaning that, given the model and prior assumptions, there is a 95% posterior probability that the `away_team_fouls` coefficient falls between 1.1 and 1.3.

Game Context:

- `overtime` (4.849), each overtime period will add about 4.8 total fouls. This makes sense because there is extra play time. A frequentist confidence interval would say that we are 95% confident the overtime predictor increases `total_fouls` between 4.375 and 5.324, holding all other variables fixed. A Bayesian credible interval would say that with a probability 0.95, the overtime predictor increases `total_fouls` between 4.4 and 5.3, holding all other variables fixed. Our interval estimate does not include zero, we can say that our predictor is statistically significant.
- `late_season` (-1.562), games in the last two weeks, they will have 1.6 fewer total fouls compared to other games. We are 95% confident the `late_season` predictor decreases `total_fouls` between 1.985 and 1.140, holding all other variables fixed. Our interval estimate does not include zero, we can say that our predictor is statistically significant.

Season Dummies (Reference Level | 2017-18):

- `season_2018_2019` (-5.521), Compared with 2017-18, about 5.5 fewer predicted fouls per game, holding all other variables fixed. The 95% confidence interval is -6.16 to -4.89. It excludes zero, so the difference is statistically significant ($p < 0.001$).

- [season_2019_2020](#) (-4.745), About 4.7 fewer predicted fouls per game. The interval is -5.38 to -4.11, so the difference is statistically significant ($p < 0.001$).
- [season_2020_2021](#) (3.642), About 3.6 more predicted fouls per game. The interval is 3.04 to 4.24, so the difference is statistically significant ($p < 0.001$).
- [season_2021_2022](#) (1.707), About 1.7 more predicted fouls per game. The interval is 1.15 to 2.27, so the difference is statistically significant ($p < 0.001$).
- [season_2022_2023](#) (-0.243), About 0.24 fewer predicted fouls per game. However, the interval is -0.77 to 0.28 and includes zero, so there is no clear evidence of a difference ($p = 0.364$).
- [season_2023_2024](#) (6.7328), About 6.7 more predicted fouls per game. The interval is 6.05 to 7.42, so the difference is statistically significant ($p < 0.001$).
- [season_2024_2025](#) (7.571), About 7.6 more predicted fouls per game. The interval is 6.85 to 8.29, so the difference is statistically significant ($p < 0.001$).
- [season_2025_2026](#) (0.487), About 0.49 more predicted fouls per game. However, the interval is -0.05 to 1.02 and includes zero, so there is no clear evidence of a difference ($p = 0.073$).

The Bayesian estimates and 95% credible intervals were very similar to the OLS results. Both methods showed the same direction of association for the seasons whose intervals excluded zero. Both included zero for 2022–23 and 2025–26, showing uncertainty about whether those seasons differed from 2017–18. A credible interval describes the parameter’s probability given the data, model, and priors; a confidence interval comes from a procedure that captures the true parameter in 95% of repeated samples.

Milestone 10